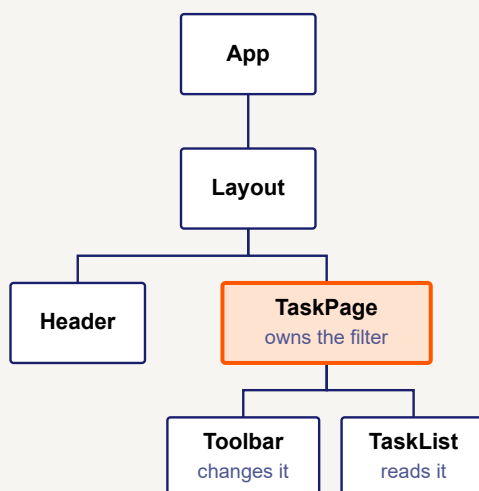


1. **Where state lives:** Where a piece of state lives · Lifting state up: data down, events up · A single source of truth · State belongs to a place in the tree (and key resets it)
2. **Prop drilling and composition:** Prop drilling, and when it actually hurts · Composition: pass components, not data
3. **Context:** Context: a value for a whole subtree · A provider component that owns the state · A custom hook for the context: `useAuth()` · Memoising: `useMemo`, `useCallback` and `memo` · What re-renders when a context value changes
4. **An authentication context:** An authentication context: user, token, log-in, log-out · Staying logged in after a reload · Where to keep the token: the security trade-off · Sharing the token with the API client
5. **Server state and stores:** Server state vs UI state · Data-fetching caches (TanStack Query) · Store libraries: Redux Toolkit and Zustand · Putting it together: a home for every piece of state

Where state lives

Where a piece of state lives

Every piece of state has one **owner**, the component that declares it: put it in the **closest common parent** of every component that reads or changes it, and no higher.



The filter lives in TaskPage: the closest parent of both components that use it.

- **Lower breaks it:** a component outside the owner's subtree cannot read the value.
- **Higher works, at a cost:** every component on the extra path passes the prop along, and each change re-renders the owner and what it renders.
- **Used by one component?** It stays local, in that component.
- **Computable from other state or props?** It is a derived value, not state: store it nowhere.

A task screen: who uses what, and where it lives

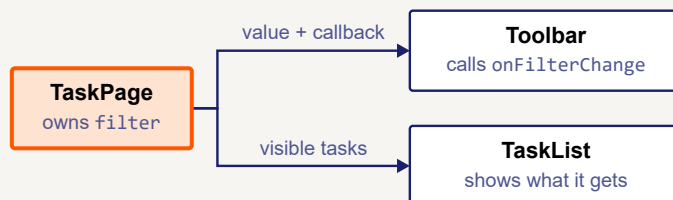
Value	Read or changed by	Lives in
The text being typed in the new-task form	TaskForm only	TaskForm (local)
The search filter	Toolbar (the input) and TaskList	TaskPage, their closest common parent
The list of tasks	Toolbar (count), TaskList, TaskItem (toggle), TaskForm (add)	TaskPage
The logged-in user	Avatar in the header, TaskList, LogoutButton	App, or a context provider near the root
How many tasks match the filter	Toolbar	Nowhere: derived during render

Example. In the state-tree tool, choose "Search filter text", then press **Place here** on Toolbar: TaskList is flagged, because it is outside Toolbar's subtree. Remove it and place it on App: it works, but Layout and TaskPage pass a prop they never use. Place it on TaskPage: everyone can reach it and nothing is passed on for nothing. Then try the first challenges.

Common mistake. Putting every piece of state at the top "so everything can reach it". It works, but each keystroke in a search box then re-renders the whole app, and every component in between has to pass props it does not care about. Start local, and move state up only when a second component needs it.

Lifting state up: data down, events up

Lifting state up moves a piece of state from a child into a parent so siblings can share it: the parent passes the **value down**, and a **callback** so the children can ask it to change.



The owner passes the value down and a callback for changes; the children never own a copy.

The value, down

As a prop: `filter={filter}`. The child never changes it (see [Props are read-only](#)).

The change, up

A callback prop named `on` + the event: `onFilterChange(text)`, `onToggle(id)`, `onAdd(task)`. The owner updates its state, and React re-renders it and its children with the new value.

Setter or named function?

Passing the setter (`onFilterChange={setFilter}`) is fine when the child may set any value; a named function (`onToggle={toggleTask}`) keeps the rules in the owner.

```

function TaskPage() {
  // lifted here
  const [filter, setFilter] = useState('');
  return (
    <>
      <Toolbar filter={filter} onFilterChange={setFilter} /> /* value + callback */
      <TaskList filter={filter} /> /* value only */
    </>
  );
}

function Toolbar({ filter, onFilterChange }) {
  return <input value={filter} onChange={(e) => onFilterChange(e.target.value)} />;
}
  
```

```

import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Write the API skeleton' },
  { id: 2, title: 'Add the log-in form' },
  { id: 3, title: 'Write the README' },
];

function Toolbar({ filter, onFilterChange, shown }) {
  return (
    <label>
      Search{' '}
      <input value={filter} onChange={(e) => onFilterChange(e.target.value)} />
      {' '}{shown} shown
    </label>
  );
}
  
```

```

    </label>
  );
}

function TaskList({ tasks }) {
  return <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}

function TaskPage() {
  // lifted: both children need it
  const [filter, setFilter] = useState('');
  const visible = TASKS.filter((t) => t.title.toLowerCase().includes(filter.toLowerCase()));
  return (
    <section>
      <Toolbar filter={filter} onFilterChange={setFilter} shown={visible.length} />
      <TaskList tasks={visible} />
    </section>
  );
}

function App() {
  return <TaskPage />;
}

```

Example. The box lifts the filter into TaskPage. Toolbar is a controlled input that reports each keystroke with onFilterChange; TaskPage derives the visible tasks and passes them to TaskList. Type in the Try it box: both children update together, because they read the same state.

Common mistake. Lifting the value but not the change: the parent passes filter down, but the child keeps calling its own local setFilter. The input then shows what was typed while the list filters by the parent's old value. When state moves up, the child loses its useState and reports changes through the callback.

A single source of truth

Each fact the interface shows is stored **once**; every other place receives it (as a prop or from a context) or computes it, because two stored copies of the same fact end up disagreeing.

How copies sneak in

- **Copying a prop into state:** useState(props.count) reads the prop once and ignores later changes (see [State, props and derived values](#)).
- **Storing what you could compute:** a count, a filtered list, an "is valid" flag (see [Don't store what you can compute](#)).
- **Two components declaring the "same" state,** because each needed it before anyone lifted it.

```

import { useState } from 'react';

function Counter({ initial }) {
  // a second copy of "how many tasks"
  const [count] = useState(initial);
  return <p>Header says: {count} tasks</p>;
}

function App() {
  const [tasks, setTasks] = useState(['Buy milk', 'Call Ana']);

  function add() {
    setTasks([...tasks, 'Task ' + (tasks.length + 1)]);
  }

  return (
    <main>
      <Counter initial={tasks.length} />
      <p>List says: {tasks.length} tasks</p>
      <button onClick={add}>Add a task</button>
    </main>
  );
}

```

```

    <ul>{tasks.map((t) => <li key={t}>{t}</li>)}</ul>
  </main>
);
}

```

Example. The box has the bug on purpose: Counter copies `tasks.length` into its own state. Add a task: the list says 3, the header still says 2, because `useState` read its argument only once. Fix it: delete the `useState` line in Counter and render `{count}` from a prop, `<Counter count={tasks.length} />`.

Common mistake. Keeping the copies and "syncing" them with an effect: `useEffect(() => setCount(tasks.length), [tasks])`. It renders once with the wrong number, then again with the right one, and every new copy needs another effect. If a value can be read from the owner or computed, it is not state.

State belongs to a place in the tree (and key resets it)

React keeps a component's state as long as the **same component type** stays at the **same position** in the tree: changing its props does not reset it; giving it a different `key` does.

Same type, same position

The state is kept, whatever the props.

A different key

A different component to React: the old instance and its state are thrown away and a fresh one mounts. `<EditTask key={task.id} task={task} />` starts a new draft for every task, with no effect.

A different type in the same place

`{isAdmin ? <AdminPanel /> : <UserPanel />}` resets the state.

A different position

Moving a component, inside a new wrapper `<div>` for example, loses its state.

```

import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Buy milk' },
  { id: 2, title: 'Call Ana' },
];

function EditTask({ task }) {
  // a deliberate copy: the edit in progress
  const [draft, setDraft] = useState(task.title);
  return (
    <p>
      Editing #{task.id}:{' '}
      <input value={draft} onChange={(e) => setDraft(e.target.value)} aria-label="Title" />
    </p>
  );
}

function App() {
  const [selectedId, setSelectedId] = useState(1);
  const [withKey, setWithKey] = useState(false);
  const task = TASKS.find((t) => t.id === selectedId);

  return (
    <main>
      {TASKS.map((t) => (
        <button key={t.id} onClick={() => setSelectedId(t.id)}>{t.title}</button>
      ))}
      <label>
        <input type="checkbox" checked={withKey} onChange={(e) => setWithKey(e.target.checked)} />
        {' use key={task.id}'}
      </label>
      {withKey ? <EditTask key={task.id} task={task} /> : <EditTask task={task} />}
    </main>
  );
}

```

Example. The box edits the selected task in a form that keeps a draft. Type something, then pick the other task: without a key the old draft stays. Tick the key checkbox and repeat: each task gets a fresh form.

Common mistake. Resetting the form with an effect: `useEffect(() => setDraft(task.title), [task.id])`. The first render after the switch still shows the old draft, then a second render fixes it, and every new field needs another line. `key={task.id}` resets all of the form's state at once, before anything is shown.

Prop drilling and composition

Prop drilling, and when it actually hurts

Prop drilling is passing a prop through components that do not use it, only so it reaches one further down: a level or two is normal and clear; it hurts when many layers forward many props.



Two components carry a value they never use, only to reach the one that does.

When it starts to hurt

- **A long chain:** four or five components.
- **Several values travelling together:** `user`, `onLogout`, `theme`, `onToggleTheme...`
- **Generic middle components:** a `Layout` or `Card` reused elsewhere, which should not know about users.
- **Every data change edits every signature** on the way. Each of these costs reading and changing time, not speed.

Drilling: fine or painful?

Situation	Verdict
A parent passes <code>task</code> and <code>onToggle</code> to its child	Not drilling: the child uses them
One middle component forwards one prop	Fine: explicit and easy to follow
A generic <code>Layout</code> forwards <code>user</code> to the header and the page	A smell: try composition
The theme or the user is read in a dozen places	A good case for context

```

function App() {
  const [user, setUser] = useState({ name: 'Ana' });
  return <Layout user={user} onLogout={() => setUser(null)} />;
}
// uses neither
function Layout({ user, onLogout }) {
  return <><Header user={user} onLogout={onLogout} /><TaskPage user={user} /></>;
}
// uses neither
function Header({ user, onLogout }) {
  return <header><Avatar user={user} /><LogoutButton onLogout={onLogout} /></header>;
}
// uses it only to pass it on
function TaskPage({ user }) {
  return <TaskList user={user} />;
}
  
```

Example. In the state-tree tool, choose "The logged-in user" and place it on **App** with Props: the tree marks `Layout`, `Header` and `TaskPage` as "only passes on". That is the challenge "Spot the drilling".

Common mistake. Reaching for context (or a store library) the first time a prop goes through one extra component. Context hides where a value comes from, makes components depend on a provider being present, and can re-render more than you expect. One or two levels of explicit props are easier to read and to test.

Composition: pass components, not data

Instead of passing data through a layout so it can build its content, let the owner of the data **build the content itself** and pass the finished elements in as `children` (or other props): the layout no longer needs to know about the data.

children One area: the content between the tags.

Slots Several areas, as props that hold elements: `<Layout header={<Header user={user} />} sidebar={<Nav />}>`. Layout, Card, Modal or Page become shells that never change when the data does.

```
import { useState } from 'react';

// knows nothing about users
function Layout({ header, children }) {
  return (
    <div className="layout">
      <header>{header}</header>
      <main>{children}</main>
    </div>
  );
}

function Avatar({ user }) {
  return <strong>{user.name}</strong>;
}

function TaskPage({ user }) {
  return <p>{user.name}'s tasks go here.</p>;
}

function App() {
  const [user, setUser] = useState({ name: 'Ana' });
  const other = user.name === 'Ana' ? 'Leo' : 'Ana';
  const header = (
    <>
      <Avatar user={user} />{' '}
      <button onClick={() => setUser({ name: other })}>Switch to {other}</button>
    </>
  );
  return (
    <Layout header={header}>
      <TaskPage user={user} />
    </Layout>
  );
}
```

Example. In the Try it box, Layout takes a header slot and children; it never sees user. App owns the user and passes it straight to Avatar and TaskPage. Switch the user: both update, and Layout's code did not have to change.

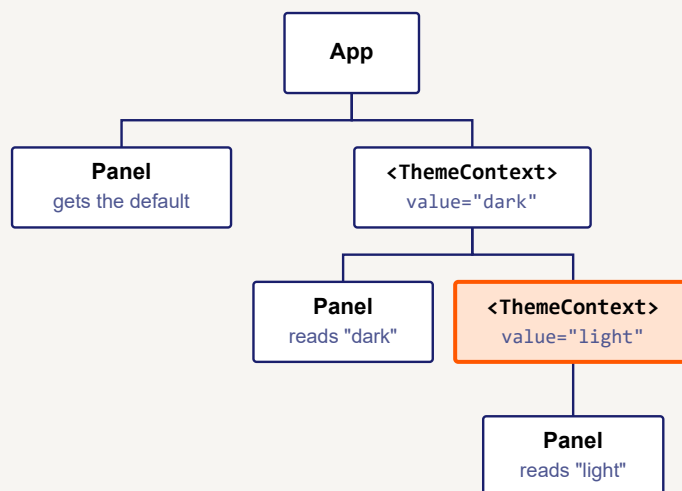
Common mistake. Thinking composition means "the Layout must render TaskPage itself". As soon as Layout writes `<TaskPage />` in its own JSX, it is the one that must provide TaskPage's props, and drilling is back. Let the owner of the data write the tag and hand the element over.

Context

Context: a value for a whole subtree

A **context** lets a component provide a value to **every component below it**, at any depth, without passing

props: `createContext(default)` makes one, a provider sets the value for its subtree, and `useContext` reads it.



Each reader gets the value of the nearest provider above it, or the default.

`createContext('light')`

Creates the channel. Its argument is the **default**, used only by a component with **no provider above it**. Create each context once, at the top level of a module (often its own file), and export it.

`<ThemeContext value="dark">`

The provider. In React 19 the context object is itself the provider; React 18 code writes `<ThemeContext.Provider value="dark">`, which React 19 still accepts.

`useContext(ThemeContext)`

Reads the value of the **nearest** provider above the calling component.

```

import { createContext, useContext } from 'react';

const ThemeContext = createContext('light');
// → 'light' is the default: no provider above

function Badge() {
  // the nearest provider above wins
  const theme = useContext(ThemeContext);
  return <span className={'badge ' + theme}>{theme}</span>;
}

// no theme prop anywhere
function Panel({ title }) {
  return <p>{title}: <Badge /></p>;
}

function App() {
  return (
    <>
      <Panel title="Outside any provider" />
      <ThemeContext value="dark">
        <Panel title="Inside a dark provider" />
        <ThemeContext value="light">
          <Panel title="Inside a nested light provider" />
        </ThemeContext>
      </ThemeContext>
    </>
  );
}

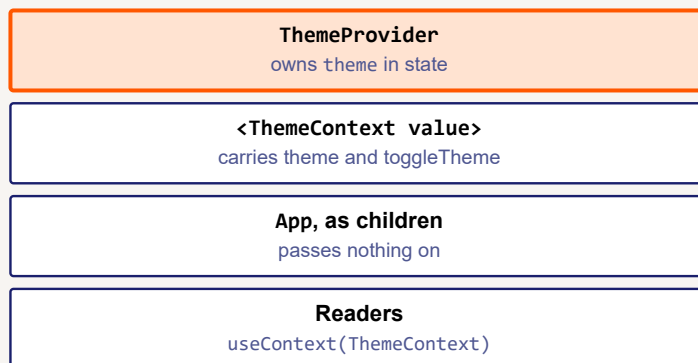
```

Example. The box has one context and three panels. The first is outside any provider and gets the default; the second is inside a "dark" provider; the third is inside a nested "light" provider, the nearest one. No component receives a theme prop.

Common mistake. Expecting the default value of `createContext` to be shared state. It is only what a component sees when there is **no provider above it**, usually because the provider was forgotten or placed too low. Many teams use `createContext(null)` and a hook that throws, so a missing provider fails loudly instead of quietly using a default (see [A custom hook for the context](#)).

A provider component that owns the state

The usual pattern: one component, for example `ThemeProvider`, declares the state, renders the context provider with the state and the functions that change it as its value, and renders `{children}` inside it.



One component owns the state; the context carries it to every reader below.

- **Expose intentions, not raw setters**, when there are rules: `toggleTheme()`, `logIn(email, password)`, `logOut()`. The provider is then the one place that knows how the state changes, and later how it is saved or sent to a server.
- **Wrap once, near the root**: `<ThemeProvider><App /></ThemeProvider>`, usually in `main.jsx`.
- `{children}` **keeps it cheap**: App was created outside the provider, so a state change re-renders only the components that read the context (see [What re-renders when a context value changes](#)).

```
// ThemeContext.jsx
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));
  return <ThemeContext value={{ theme, toggleTheme }}>{children}</ThemeContext>;
}

// main.jsx
createRoot(document.getElementById('root')).render(
  <ThemeProvider><App /></ThemeProvider>
);
```

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext(null);

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));
  return <ThemeContext value={{ theme, toggleTheme }}>{children}</ThemeContext>;
}

function ThemeToggle() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return <button onClick={toggleTheme}>Switch to {theme === 'light' ? 'dark' : 'light'}</button>;
}
```



```
// passes nothing on
function Sidebar() {
  return <aside><ThemeToggle /></aside>;
}

function Page() {
  const { theme } = useContext(ThemeContext);
  return (
    <div className={'page ' + theme}>
      <h2>Tasks</h2>
      <Sidebar />
    </div>
  );
}

function App() {
  return (
    <ThemeProvider>
      <Page />
    </ThemeProvider>
  );
}
```

Example. In the Try it box, ThemeToggle sits inside Sidebar, which passes nothing on, yet it reads and changes the theme owned by ThemeProvider. Page reads the same value to choose its colours.

Common mistake. Calling useContext(ThemeContext) in the same component that renders the provider and expecting the new value. useContext looks **above** the calling component, so the provider's own component never sees its own provider: it already has the state in a variable; use that.

A custom hook for the context: useAuth()

Wrap useContext **in a custom hook such as** useAuth() **that throws a clear error when there is no provider: components import the hook, not the context object.**

- **An interface, not only a check:** export AuthProvider and useAuth and keep the context object private. Readers cannot depend on how the value is built, so you can later split the context, add memoisation or move to a library without touching a single component.
- **One file per context:** auth/AuthContext.jsx with the context (not exported), AuthProvider and useAuth. Vite's Fast Refresh prefers files that export only components; if the linter complains, move the hook to auth/useAuth.js.

```
const AuthContext = createContext(null);

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (ctx === null) {
    throw new Error('useAuth must be used inside <AuthProvider>');
  }
  return ctx;
}
```

```
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logIn = (name) => setUser({ name });
  const logOut = () => setUser(null);
  return <AuthContext value={{ user, logIn, logOut }}>{children}</AuthContext>;
}
```

```
function useAuth() {
  const ctx = useContext(AuthContext);
  if (ctx === null) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}

function Greeting() {
  const { user, logIn, logOut } = useAuth();
  if (!user) return <button onClick={() => logIn('Ana')}>Log in as Ana</button>;
  return <p>Hello, {user.name} <button onClick={logOut}>Log out</button></p>;
}

// Try: render <Greeting /> outside <AuthProvider> and read the error.
function App() {
  return (
    <AuthProvider>
      <Greeting />
    </AuthProvider>
  );
}
```

Example. The box works as written. Then move `<Greeting />` out of `<AuthProvider>` (put it after the closing tag, inside a fragment `<>...</>`): the preview shows the hook's error message instead of a confusing crash.

Common mistake. Exporting the context object and calling `useContext(AuthContext)` in every component. It works until the first component outside the provider (a test, a storybook page, a new route) receives `null`. A hook gives one place for the check and one name to search for.

Memoising: `useMemo`, `useCallback` and `memo`

Memoising keeps a result from the last render while its inputs are unchanged: `useMemo` keeps a computed value, `useCallback` keeps a function, and `memo` lets a component skip a re-render when its props are equal.

Three tools

You write	It keeps	Until
<code>useMemo(() => compute(a, b), [a, b])</code>	The value the function returned	a or b changes
<code>useCallback(fn, [a])</code>	The function itself: the same object	a changes
<code>memo(TaskList)</code>	The component's last output	A prop changes (compared one by one with <code>Object.is</code>)

- **They work together:** `memo` compares props by identity, so a child wrapped in `memo` still re-renders when the parent passes a new function or object every time. `useCallback` and `useMemo` keep those props the same.
- **Never for correctness:** the code must work without them; React may throw a memoised value away.
- **Premature by default:** add them when a render is measurably slow (the React DevTools Profiler shows it) or when identity matters, not around every function.

```
import { memo, useCallback, useState } from 'react';

let renders = 0;
const TaskList = memo(function TaskList({ onSelect }) {
  // counts the times React really ran it
  renders += 1;
  return (
    <>
      <p>TaskList rendered {renders} times</p>
      <ul>
        {[ 'Buy milk', 'Call Ana' ].map((t) => (
          <li key={t}><button onClick={() => onSelect(t)}>{t}</button></li>
        ))}
      </ul>
    </>
  );
});
```

```

    );
  });

function App() {
  const [text, setText] = useState('');
  const [selected, setSelected] = useState('none');
  const [stable, setStable] = useState(false);

  // a new function every render
  const fresh = (t) => setSelected(t);
  // the same function every render
  const kept = useCallback((t) => setSelected(t), []);
  const onSelect = stable ? kept : fresh;

  return (
    <main>
      <label>
        <input type="checkbox" checked={stable} onChange={(e) => setStable(e.target.checked)} />
        { ' ' }Stable callback (useCallback)
      </label>
      <p><input value={text} onChange={(e) => setText(e.target.value)} placeholder="Type here" aria-label="Note" />
    </p>
      <p>Selected: {selected}</p>
      <TaskList onSelect={onSelect} />
    </main>
  );
}

```

Example. In the Try it box, TaskList is wrapped in memo and shows how many times it rendered. Type in the field: App re-renders on every key, onSelect is a new function each time, and the count climbs. Tick **Stable callback** and type again: useCallback keeps the same function, the props are equal, and memo skips the list.

Common mistake. Wrapping every function in useCallback and every value in useMemo "for performance". Each one costs memory and a dependency array to keep right, and saves nothing unless something compares the identity. A wrong dependency array is worse than none: it keeps a stale value.

What re-renders when a context value changes

When the provider's value changes (compared with Object.is), **every component that reads that context re-renders, even inside memo**; a new object literal on every render counts as a change, so memoise the value with useMemo.

Three habits keep context cheap

- **Wrap with {children}**: <AuthProvider>{children}</AuthProvider>, so a state change re-renders only the consumers.
- **Split by speed of change**: AuthContext and ThemeContext, not one AppContext.
- **Keep fast data out**: never put the mouse position, or every keystroke of a big form, in a context read by many components.

```

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logIn = useCallback((u) => setUser(u), []);
  const logOut = useCallback(() => setUser(null), []);
  const value = useMemo(() => ({ user, logIn, logOut }), [user, logIn, logOut]);
  return <AuthContext value={value}>{children}</AuthContext>;
}

```

```

import { createContext, useContext, useMemo, useState, memo } from 'react';

const AuthContext = createContext(null);

const UserBadge = memo(function UserBadge() {
  const { user } = useContext(AuthContext);
  console.log('UserBadge rendered');
  return <p>Signed in as {user.name}</p>;
});

```

```

});

function App() {
  const [clicks, setClicks] = useState(0);
  const [user, setUser] = useState({ name: 'Ana' });
  const [stable, setStable] = useState(false);

  // a new object every render
  const fresh = { user, setUser };
  // same object until user changes
  const memoised = useMemo(() => ({ user, setUser }), [user]);
  const value = stable ? memoised : fresh;

  return (
    <AuthContext value={value}>
      <label>
        <input type="checkbox" checked={stable} onChange={(e) => setStable(e.target.checked)} />
        { ' ' }Memoise the value (useMemo)
      </label>
      <p><button onClick={() => setClicks((c) => c + 1)}>Unrelated click: {clicks}</button></p>
      <UserBadge />
    </AuthContext>
  );
}

```

Example. In the Try it box, UserBadge is wrapped in memo and reads the context. Press **Unrelated click** and watch the console: App re-renders, the value object is new, and the badge re-renders although the user did not change. Tick "Memoise the value" and click again: the badge stays quiet.

Common mistake. Putting everything into one big AppContext "to keep it simple". Every consumer then re-renders whenever any part changes: typing in a search field stored there re-renders the header, the sidebar and every list item. Separate contexts for separate concerns, and local state for anything only one area uses.

An authentication context

An authentication context: user, token, log-in, log-out

An AuthProvider **owns who is logged in: the user and the token in state, an async logIn(email, password) that calls the API and stores both, and logOut() that forgets them; components read it all with useAuth().**

State	The token and the public user fields. Never the password.
logIn(email, password)	Posts the credentials (see Logging in from the front end) and stores the token and the user. On failure it throws , so the form that called it shows the message; the provider knows nothing about forms. Reading the form: Uncontrolled inputs: defaultValue and FormData .
logOut()	Clears both, and later the saved copy in storage.
Derived values	isLoggedIn = token !== null, isAdmin = user?.role === 'admin': computed, not stored.

```

import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [token, setToken] = useState(null);
  const [user, setUser] = useState(null);

  async function logIn(email, password) {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password }),
    });
  }
}

```

```

    const data = await res.json();
    // the form shows it
    if (!res.ok) throw new Error(data.error);
    setToken(data.token);
    setUser(data.user);
  }
  const logOut = () => { setToken(null); setUser(null); };

  return <AuthContext value={{ user, token, logIn, logOut }}>{children}</AuthContext>;
}
const useAuth = () => useContext(AuthContext);

function LoginForm() {
  const { logIn } = useAuth();
  const [error, setError] = useState('');
  async function handleSubmit(e) {
    e.preventDefault();
    const form = new FormData(e.target);
    try { await logIn(form.get('email'), form.get('password')); } catch (err) { setError(err.message); }
  }
  return (
    <form onSubmit={handleSubmit}>
      <input name="email" defaultValue="ana@example.com" aria-label="Email" />
      <input name="password" type="password" placeholder="password123" aria-label="Password" />
      <button>Log in</button> {error && <span role="alert">{error}</span>}
    </form>
  );
}

function Header() {
  const { user, token, logOut } = useAuth();
  if (!user) return <LoginForm />;
  return <p>{user.name} ({user.role}) · token <code>{token}</code> <button onClick={logOut}>Log out</button></p>;
}

function App() {
  return <AuthProvider><Header /></AuthProvider>;
}

```

Example. The box logs in against the mock API. Use ana@example.com (an admin) or leo@example.com (a student) with the password password123; try a wrong password to see the error come back from logIn to the form.

Common mistake. Storing the password, or the whole log-in response "just in case", in the context. Once logged in, the front end needs the token and the public user fields only. A password kept in memory can leak through logs, error reports or dev tools; drop it as soon as the request has been sent.

Staying logged in after a reload

State lives in memory, so a reload logs the user out: to keep the session, the provider **restores** the token (and the user) from `localStorage` **once**, when it is created, and **saves** them whenever they change.

Restore: a lazy initialiser

`useState(() => readSaved())` runs once, before the first render, so the app never flashes a logged-out screen. The same idea as a reusable hook: [A custom hook for persistence: useLocalStorage](#).

Save: an effect

On every change of the session: write it when it is set, remove it when it is null.

One key, JSON and try/catch

Storage holds only strings and can be blocked, full or hold garbage, and the app must still start (logged out). One key, `session`, for the token and the user, so they cannot drift apart.

```

function AuthProvider({ children }) {
  // restore once
  const [session, setSession] = useState(() => {
    try { return JSON.parse(localStorage.getItem('session')) ?? null; }
  });

```

```

    catch { return null; }
  });

  // save on every change
  useEffect(() => {
    try {
      if (session) localStorage.setItem('session', JSON.stringify(session));
      else localStorage.removeItem('session');
    } catch { /* storage blocked: the session lasts until reload */ }
  }, [session]);
  // logIn: setSession({ token: data.token, user: data.user }) · logOut: setSession(null)
  ...
}

```

```

import { createContext, useContext, useEffect, useState } from 'react';

// The preview blocks localStorage, so this falls back to memory. Your app uses localStorage.
const memory = {};
const storage = {
  get: (k) => { try { return localStorage.getItem(k); } catch { return memory[k] ?? null; } },
  set: (k, v) => { try { localStorage.setItem(k, v); } catch { memory[k] = v; } },
  remove: (k) => { try { localStorage.removeItem(k); } catch { delete memory[k]; } },
};

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  // restore once, on load
  const [session, setSession] = useState(() => {
    const saved = storage.get('session');
    console.log(saved ? 'restored the saved session' : 'no saved session');
    return saved ? JSON.parse(saved) : null;
  });
  useEffect(() => {
    if (session) storage.set('session', JSON.stringify(session));
    else storage.remove('session');
  }, [session]);
  const logIn = () => setSession({ token: 'student-token', user: { name: 'Leo' } });
  const logOut = () => setSession(null);
  return <AuthContext value={{ user: session?.user ?? null, logIn, logOut }}>{children}</AuthContext>;
}

function Status() {
  const { user, logIn, logOut } = useContext(AuthContext);
  if (!user) return <p>Logged out <button onClick={logIn}>Log in as Leo</button></p>;
  return <p>Logged in as {user.name} <button onClick={logOut}>Log out</button></p>;
}

function App() {
  // a new key = a fresh provider, like a page reload
  const [boot, setBoot] = useState(0);
  return (
    <>
      <button onClick={() => setBoot((b) => b + 1)}>Simulate a reload</button>
      <AuthProvider key={boot}><Status /></AuthProvider>
    </>
  );
}

```

Example. The preview frame blocks `localStorage`, so the box falls back to a variable in memory (your app uses the real storage). Log in, then press **Simulate a reload**: the provider is created again (a new key), its initialiser finds the saved session and you stay logged in. Log out and reload: the session is gone.

Common mistake. Restoring in an effect: `useEffect(() => setToken(localStorage.getItem('token')), [])`. The first render runs with `token = null`, so a protected page redirects to the log-in page before the effect restores the token, and the user lands on the log-in screen although they are logged in. Read storage in the lazy initialiser, before the first render.

Where to keep the token: the security trade-off

A token in **memory** is lost on reload; in **localStorage** it survives but any script on the page can read it; in an **httpOnly cookie** set by the server JavaScript cannot read it at all, but cookies bring their own rules (CSRF, CORS credentials).

Three places for the session

Where	Survives a reload	Readable by any script on the page	Sent automatically	Watch out for
Memory (state, context)	No	Only while the page is open	No	Users logged out on every reload
localStorage	Yes	Yes	No (you add the header)	XSS steals it
httpOnly cookie	Yes	No	Yes, by the browser	CSRF, CORS credentials

Cookies have their own rules

- **CSRF:** the browser attaches cookies automatically, so another site can make it send a request with them; `SameSite=Lax` or `Strict` and checking the `Origin` header defend against it.
- **CORS credentials:** a front end on another origin needs credentials: `'include'` and a server that allows credentials (see [CORS with credentials](#)).
- **XSS can still use the cookie** by sending requests from the open page; it just cannot steal it for later.

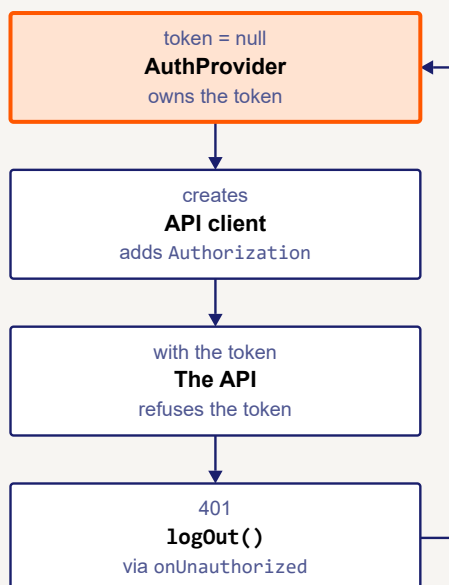
Example. With a token in `localStorage`, open the browser's dev tools on your own app, Application tab, Local Storage: the token is right there in plain text, for you and for any script running in the page. With an `httpOnly cookie`, the Cookies panel shows it, but `document.cookie` in the console does not include it.

Common mistake. Believing that `localStorage` is "encrypted" or private to your code, or that `sessionStorage` is safer against XSS. Both are readable by any script running on the page; `sessionStorage` only differs in lasting until the tab closes. The protection against token theft is preventing XSS, and `httpOnly` cookies when that is not enough.

Sharing the token with the API client

The **API client** needs the current token, and a refused token must log the user out: let the auth provider build

the client with the token and its own `logOut` as the `onUnauthorized` handler, and expose it through the context.



Every request gets the current token, and a refused token logs out in one place.

Built in the provider

`useMemo(() => createClient({ token, onUnauthorized: logOut })), [token])`, exposed in the context: when the token changes, components get a client with the new one.

A module-level client

With a `getToken()` callback the provider registers once: suits code outside React, such as a router loader.

```

function AuthProvider({ children }) {
  const [session, setSession] = useState(restoreSession);
  const logOut = useCallback(() => setSession(null), []);
  const token = session?.token ?? null;
  const api = useMemo(() => createClient({ token, onUnauthorized: logOut })), [token, logOut]);
  const value = useMemo(() => ({ user: session?.user ?? null, api, logOut /* , logIn */ }), [session, api, logOut]);
  return <AuthContext value={value}>{children}</AuthContext>;
}

```

```

import { createContext, useContext, useMemo, useState } from 'react';

function createClient({ token, onUnauthorized }) {
  return async function api(path, { method = 'GET', body } = {}) {
    const headers = { 'Content-Type': 'application/json' };
    if (token) headers.Authorization = 'Bearer ' + token;
    const res = await fetch('/api' + path, { method, headers, body: body && JSON.stringify(body) });
    if (res.status === 401 && token) { onUnauthorized(); throw new Error('Session expired: log in again'); }
    const data = await res.json();
    if (!res.ok) throw new Error(data.error);
    return data;
  };
}

const AuthContext = createContext(null);

function AuthProvider({ children }) {
  // as if restored days later
  const [token, setToken] = useState('expired-token');
  const logOut = () => setToken(null);
  const api = useMemo(() => createClient({ token, onUnauthorized: logOut })), [token];
  async function logIn() {
    const data = await api('/auth/login', { method: 'POST', body: { email: 'ana@example.com', password: 'password123' } });
    setToken(data.token);
  }
}

```



```

    }
    return <AuthContext value={{ token, api, logIn }}>{children}</AuthContext>;
  }

function CreateUser() {
  const { token, api, logIn } = useContext(AuthContext);
  const [msg, setMsg] = useState('');
  async function create() {
    try {
      const u = await api('/users', { method: 'POST', body: { name: 'Iris', email: 'iris' + Date.now() + '@example.com', password: 'longpassword' } });
      setMsg('Created user ' + u.id);
    } catch (err) { setMsg(err.message); }
  }
  return (
    <section>
      <p>Token: <code>{token ?? '(none)'}</code> {!token && <button onClick={logIn}>Log in as Ana</button>}</p>
      <button onClick={create}>Create a user (admins only)</button> <output>{msg}</output>
    </section>
  );
}

function App() {
  return <AuthProvider><CreateUser /></AuthProvider>;
}

```

Example. The box starts with a token "restored from storage days ago" that the server no longer accepts. Press **Create a user**: the client gets 401 with a token, calls the provider's `logout`, and the token disappears. Log in as Ana (a real call to the mock API) and create again: 201. Log out and try once more: 401 without a token is just an error, not a log-out.

Common mistake. Reading the token from `localStorage` inside the client on every request. It looks like a single source, but now there are two: the context says "logged out" while storage still has the token (or the other way round after a failed write), and logging out in one tab behaves differently from another. The provider owns the session; storage is only its backup.

Server state and stores

Server state vs UI state

UI state belongs to the interface (is the menu open, what is typed, which tab is active); **server state** is a copy of data that lives on a server (the tasks, the users): it can go stale, is shared by many screens and must be refetched after changes.

Which kind is it?

Value	Kind	Where it goes
Is the side menu open	UI	Local state
The search text	UI	Lifted state, or the URL (<code>?search=</code>)
The logged-in user and token	UI (the session)	The auth context
The tasks from <code>GET /api/tasks</code>	Server	A fetching hook or a query cache
The task being edited (unsaved)	UI (a draft)	Local state in the form

```

import { useEffect, useState } from 'react';

// Each call keeps its OWN copy of the server's tasks.
function useTasks() {
  const [tasks, setTasks] = useState([]);
  async function reload() {
    const res = await fetch('/api/tasks');
    setTasks(await res.json());
  }
  useEffect(() => { reload(); }, []);
  return { tasks, reload };
}

```

```

}

function TaskCount() {
  // request 1
  const { tasks } = useTasks();
  return <p>Header: {tasks.length} tasks</p>;
}

function TaskList() {
  // request 2, same URL
  const { tasks, reload } = useTasks();
  async function add() {
    await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'New task ' + tasks.length }),
    });
  };
  // refreshes THIS copy only
  reload();
}

return (
  <>
    <p>List: {tasks.length} tasks</p>
    <button onClick={add}>Add a task</button>
  </>
);
}

function App() {
  return <><TaskCount /><TaskList /></>;
}

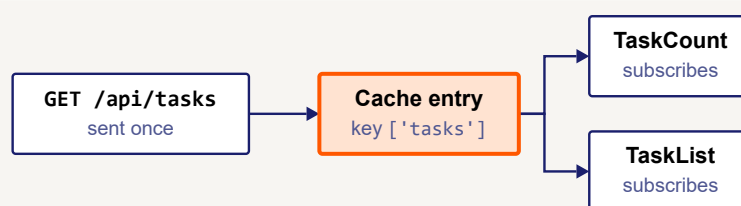
```

Example. In the Try it box, the header and the list both call the same `useTasks()` hook. The console shows **two** identical requests. Add a task from the list: the list refetches its copy, the header keeps its old count. Two copies of server state, two truths.

Common mistake. Loading the tasks once into a global context "so every screen has them" and never refetching. The context now holds a snapshot that ages: another user's changes never appear, and each write needs hand-written code to patch the global list. Server data needs a plan for staleness, not just a place to live.

Data-fetching caches (TanStack Query)

A **data-fetching cache** keeps one copy of each server response under a **key** (`['tasks']`), shares it with every component that asks for the same key, removes duplicate requests and refetches when you **invalidate** the key after a write.



One key, one request, one copy shared by every subscriber.

- **What the libraries add:** `isPending` / `isError` / data states, retries, refetching when the window regains focus, keeping old data while a new page loads, cancelling, and optimistic updates (see [Optimistic updates](#)).
- **Parameters are part of the key:** `['tasks', { search, page }]` caches each combination separately.
- **When to reach for one:** several screens share the same data, the same refetch-after-write code keeps coming back, or going back to a page should feel instant. A hook per resource plus lifting covers small and medium apps.

```
// With TanStack Query (wrap the app once in <QueryClientProvider client={queryClient}>)
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function useTasks(search) {
  const { api } = useAuth();
  return useQuery({ queryKey: ['tasks', { search }], queryFn: () => api('/tasks?search=' +
    encodeURIComponent(search)) });
}

function useAddTask() {
  const { api } = useAuth();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (title) => api('/tasks', { method: 'POST', body: { title } }),
    // every tasks list refetches
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['tasks'] }),
  });
}

// in a component
const { data: tasks, isPending, isError, error } = useTasks(search);
```

```
import { useEffect, useState } from 'react';

// A toy query cache: one entry per key, shared by every component that asks for it.
// key → { data, promise, listeners }
const cache = new Map();
function entry(key) {
  if (!cache.has(key)) cache.set(key, { data: undefined, promise: null, listeners: new Set() });
  return cache.get(key);
}
function fetchKey(key, fn) {
  const e = entry(key);
  if (!e.promise) e.promise = fn().then((data) => { e.data = data; e.promise = null; e.listeners.forEach((l) =>
    l(data)); });
  return e.promise;
}
function useQuery(key, fn) {
  const e = entry(key);
  const [data, setData] = useState(e.data);
  useEffect(() => {
    e.listeners.add(setData);
    // first asker fetches, the others share the promise
    if (e.data === undefined) fetchKey(key, fn);
    return () => e.listeners.delete(setData);
  }, [key]);
  return data;
}

const getTasks = () => fetch('/api/tasks').then((r) => r.json());

function TaskCount() {
  const tasks = useQuery('tasks', getTasks);
  return <p>Header: {tasks ? tasks.length : '...'} tasks</p>;
}

function TaskList() {
  const tasks = useQuery('tasks', getTasks);
  async function add() {
    await fetch('/api/tasks', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
    JSON.stringify({ title: 'New task' }) });
    fetchKey('tasks', getTasks);
    // → "invalidate": refetch once, notify everyone
  }
  return <><p>List: {tasks ? tasks.length : '...'} tasks</p><button onClick={add}>Add a task</button></>;
}

function App() {
```

```
return <><TaskCount /><TaskList /></>;
}
```

Example. The box is the app of [Server state vs UI state](#) with a 20-line toy cache. The console shows **one** request although two components ask for the tasks. Add a task: the cache refetches once and **both** components update.

Common mistake. Copying the query's data into useState (const [tasks, setTasks] = useState(data) and an effect to sync it). That recreates the second source of truth the cache exists to remove: the copy misses every refetch. Read data from the hook directly; for edits, keep a separate draft.

Store libraries: Redux Toolkit and Zustand

A **store** is an object outside the component tree that holds app-wide state; components **subscribe to the slice they select** and re-render only when that slice changes. Most apps of moderate size do not need one.

Zustand Small: create((set) => ({ ... }))) returns a hook; no provider needed.

Redux Toolkit Structured: slices with reducers, configureStore, a <Provider>, useSelector and useDispatch. It suits large teams that want strict conventions, and includes RTK Query, a data-fetching cache.

```
// Zustand
import { create } from 'zustand';
export const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
}));
// re-renders only when the count changes
const count = useCart((s) => s.items.length);

// Redux Toolkit
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  // Immer makes this immutable
  reducers: { add: (state, action) => { state.items.push(action.payload); } },
});
const count = useSelector((s) => s.cart.items.length);
dispatch(cartSlice.actions.add(item));
```

```
import { useSyncExternalStore } from 'react';

function createStore(initial) {
  let state = initial;
  const listeners = new Set();
  return {
    get: () => state,
    set: (change) => { state = { ...state, ...change(state) }; listeners.forEach((l) => l()); },
    subscribe: (l) => { listeners.add(l); return () => listeners.delete(l); },
  };
}

const store = createStore({ count: 0, theme: 'light' });
const useStore = (selector) => useSyncExternalStore(store.subscribe, () => selector(store.get()));

function Count() {
  const count = useStore((s) => s.count);
  console.log('Count rendered');
  return <p>count: {count}</p>;
}

function Theme() {
  const theme = useStore((s) => s.theme);
  console.log('Theme rendered');
  return <p>theme: {theme}</p>;
}
```

```
// reads nothing, so it never re-renders
function App() {
  return (
    <>
      <Count />
      <Theme />
      <button onClick={() => store.set((s) => ({ count: s.count + 1 })))>+1</button>{' '}
      <button onClick={() => store.set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })))>Theme</button>
    </>
  );
}
```

Example. The box builds a 10-line store with `useSyncExternalStore`. **Count** selects `count`, **Theme** selects `theme`. Press **+1**: only **Count** logs a render. Press **Theme**: only **Theme** does. With one context holding both, both would re-render every time.

Common mistake. Moving every piece of state into the store, including form fields and whether a dropdown is open. The store then becomes a global variable with extra steps: components depend on it for things only they use, tests need it set up, and you lose the "state lives where it is used" rule. Keep local state local even when a store exists.

Putting it together: a home for every piece of state

For each value, ask in order: is it derived, used by one component, by a few nearby, part of the URL, app-wide and slow-changing, or server data? Stop at the first yes; a store is for what is left.

1. **Can it be computed** from other state or props? Derive it during render and store it nowhere.
2. **Does only one component use it?** Local state.
3. **Do siblings, or a parent and a child, use it?** Lift it to the closest common parent and pass props (see [Where a piece of state lives](#)).
4. **Does the path go through wrappers** (Layout, Card, Page)? Composition: pass elements as children or slots.
5. **Should it survive a reload or be shareable** (the screen, the selected id, the search, the page)? The URL (see [Query strings: useSearchParams](#)).
6. **Do many components at different depths need it, and does it change rarely** (the user, the theme, the language)? A context with a provider component.
7. **Is it a copy of server data?** A fetching hook per resource, or a data-fetching cache.
8. **Is it complex client state that many components change?** A store library.

Props, composition or context: the trade-off

Props	Explicit: you can see where every value comes from. Noisy over long paths.
Composition	Still explicit, and shortens the path, but only when the middle components are wrappers.
Context	Removes the path, at the price of implicit dependencies : a component that calls <code>useAuth()</code> works only inside an <code>AuthProvider</code> , and its props no longer show what it needs. Good context values have many readers, a natural owner near the root and a low rate of change .

A home for every value in a task app

Value	Home
The number of done tasks	Derived during render
The new-task title being typed	Local state in the form
The selected task in a list + detail split	Lifted to the page (or <code>/tasks/:id</code> in the URL)
The search text and the page number	The URL query string, or lifted state
The logged-in user and token	An auth context, saved to storage
The theme	A theme context, saved to storage

Value	Home
The tasks from the server	A fetching hook per resource, or a query cache
A complex client-side editor document	A store library

Example. Try the state-tree challenges in order: keep local state local, lift the filter, remove a duplicate, spot the drilling, fix it with composition, move the theme into a context, then predict the re-renders. One check of context overkill: choose "Search filter text", switch to Context and place the provider on TaskPage. The analysis says plain props would do, because nothing sits between TaskPage and the two components that use the filter.

Common mistake. Deciding the "state management architecture" before writing the screens, or putting the search text, the selected task and the open modal in one root context "so everything can reach it". The right home comes from who uses a value, which you only know once the components exist, and a root context of fast-changing values re-renders every consumer on each keystroke. Start local, lift when needed, and add a context, cache or store when a concrete pain shows up.